FEATURE SPECIFICATION

Shelf Transfer Mod for Sons of the Forest

A client-only mod that moves a stack of one item type from one storage shelf to another in a single action, with shelves grouped by base and labelled for recognition.

> **Why this exists.** A shelf cannot be moved while it holds items, and emptying it means carrying items one at a time to another shelf. This mod removes the trips: look at a slot, press one key, pick the target shelf, confirm. The host sees the result without installing anything.

STATUS	DATE	VERSION
Draft for build	18 October 2026	1.1

PLATFORM	STACK	SCOPE
Sons of the Forest, PC (Steam)	BepInEx 6 IL2CPP · HarmonyX · IMGUI	Client-side only, shelves only

CONTENTS

What this document covers

01	Feature	pain point, behaviour, scope
02	Acceptance criteria	five conditions that define done
03	Player flow	in-game steps with screenshot
04	System architecture	modules and their boundaries
05	How it works	transfer sequence, host model, failure handling
06	Overlay	mockup, groups, amount, result
07	Shelf labels and groups	identity, storage, display
08	Tech stack	decisions and the alternative considered
09	Build order and test plan	six steps and a test matrix
10	Claude Code brief	pre-flight, risk review, kick-off prompt
11	Risks and open questions	what could change the design

SECTION 01

Feature

A client-only Sons of the Forest mod that moves a stack of one item type from a storage shelf to another shelf in one action, without walking between them.

Pain point

A shelf cannot be moved while it holds items, and the only way to empty it is to take items one at a time and carry them to another shelf. Reorganising a base means many trips.

What the mod does

When you look at a slot on a shelf (for example 6 MREs at row 1, column 1), a third prompt appears next to the game's own Drop and Take prompts. Pressing it opens an overlay that lists your other shelves by base and label. You pick a target shelf and an amount (default: all), confirm, and the items move. Both you and the host see them on the target shelf; the source slot is empty.

IN SCOPE

- Storage shelves only
- One item type per transfer (the slot you are looking at)
- Transfer all or a chosen amount
- Client-side shelf labels and base groups
- Works on a friend-hosted session with only you running the mod

OUT OF SCOPE

- Moving a shelf with its contents
- Other storage types (log holders, stick holders, drying racks)
- Any host-side component
- Sharing labels or groups with other players

SECTION 02

Acceptance criteria

The feature is done when all five hold on a friend-hosted session, with only the client running the mod.

- [ ] Looking at a specific item slot on a shelf shows a Transfer key prompt alongside the game's Drop and Take prompts
- [ ] Pressing the key opens an overlay listing labelled shelves, grouped by base
- [ ] The overlay lets you transfer all, or a set amount, of the selected item type to a chosen shelf
- [ ] After confirming, the items are on the target shelf and no longer on the source shelf
- [ ] Both client and host see the items on the target shelf, including after both reload the save

SECTION 03

Player flow

The transfer is a seven-step flow that starts from the existing shelf interaction and adds one key.



Figure 1. Looking at a stack on the top row of a shelf. The game shows **E** to take and **C** to drop. The mod draws a third prompt beside them, for example **L — Transfer**.

1. Look at an item slot on a shelf. Row and column resolve from the game's own aim target.

2. The mod draws the **L — Transfer** hint next to the game's E and C prompts.
3. Press L. The game cursor unlocks and the overlay opens.
4. The overlay lists all other shelves in the world, grouped by base, each with its label (or distance and bearing when unlabelled) and how many of this item type it can still take.
5. Pick the target shelf. Set the amount: all (default) or a number, clamped to what the target can fit.
6. Press Confirm, or Esc to cancel.
7. The items move one at a time over a few frames. The overlay reports moved and remaining, then closes.

Labelling a shelf is a side flow: look at any shelf, press the label key, type a name, press Enter. Assigning a shelf to a group works the same way from the label editor or from the overlay row. The label shows in the overlay list and as small text above the shelf when you are near it.

SECTION 04

System architecture

Six small modules sit on top of the game's own interaction layer. None of them writes to shelf state directly; the only path to the host is the same take and store calls the game fires when you press E or C.

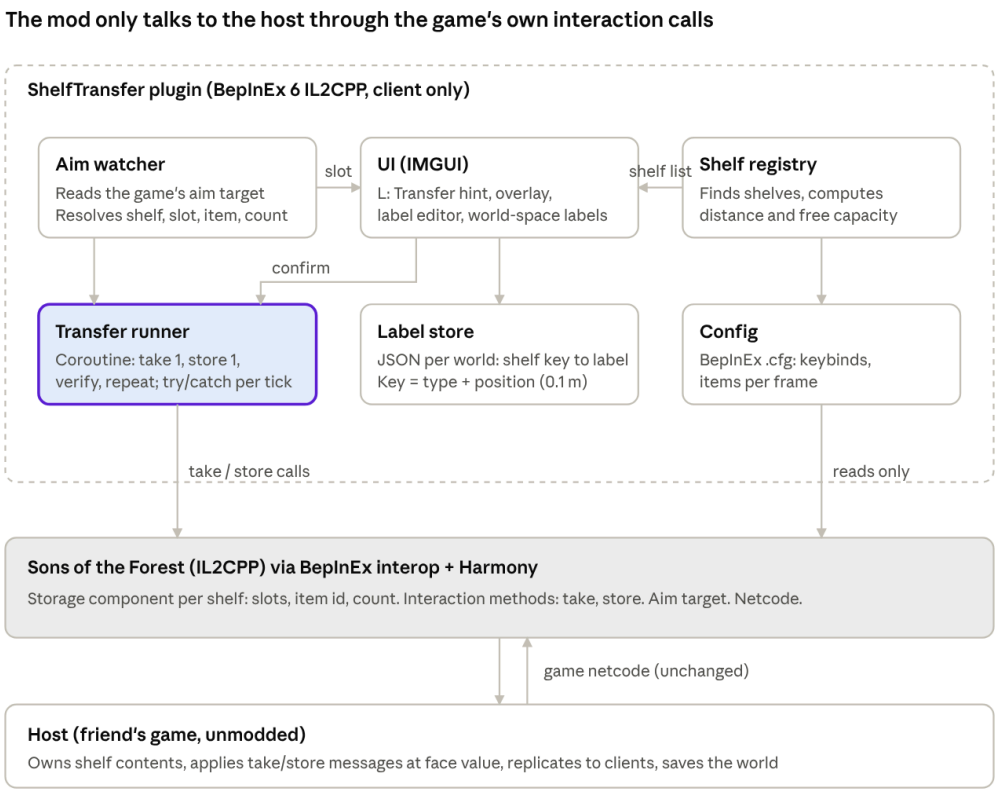


Figure 2. Mod architecture. The Transfer runner is the only module with side effects on the host; the Shelf registry and Aim watcher only read game state; UI, Label store and Config never touch the game.

MODULE	RESPONSIBILITY	DEPENDS ON
Aim watcher	Each frame, resolve what the player is looking at into shelf, slot, item type, count	Game aim target (one Harmony patch if not readable directly)
UI	L: Transfer hint, overlay with groups, label editor, world-space label text	IMGUI via injected MonoBehaviour
Shelf registry	Enumerate shelf storage components; distance, bearing, free capacity for an item type	Game storage component
Transfer runner	Coroutine: take 1 from source, store 1 to target, verify both counts, repeat; stop and report on any failure	Game take/store methods

MODULE	RESPONSIBILITY	DEPENDS ON
Label store	Load and save shelf key → { label, group } per world as JSON	File system
Config	Keybinds and items per frame	BepInEx config

SECTION 05

How it works

The transfer is N repetitions of the two actions a player already performs by hand, with a check after each, so the host never receives anything it would not receive from a vanilla player.

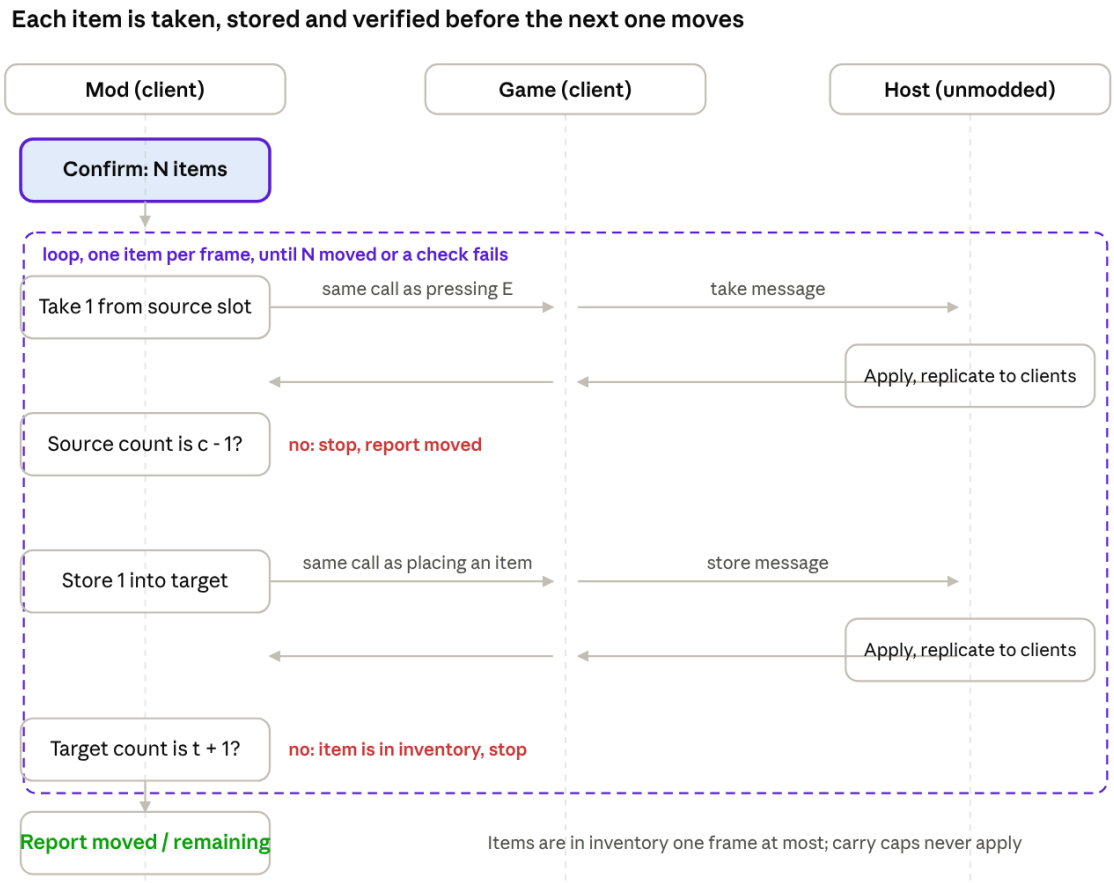


Figure 3. Transfer sequence. Each item is taken, stored and verified before the next one moves.

Host and client model

Shelf contents are host-authoritative and saved by the host; the player inventory is client-authoritative and saved by the client. The host applies take and store messages at face value

without checking inventory or player position. This is observable in the game today: a client who saves, stores ammo on a shelf, quits and rejoins keeps the ammo in inventory while the shelf also holds it.

> For the mod this means two things: calling the game's own take and store methods from the client is enough to move items on the host, and a bug in the mod would act on the host's real save. Each step is therefore verified before the next.

Why one item per frame

Spacing the calls lets each step's result arrive before the next, keeps the host from being flooded, and bounds the damage of any failure to one item. Items pass through the player's inventory for at most one frame, so per-item carry caps never apply.

Failure handling

- **Target cannot take the item type:** the overlay dims that shelf and the transfer never starts.
- **Target fills up mid-transfer:** the store call is refused, the item stays in inventory, the loop stops and reports moved and remaining.
- **Source count does not drop after a take:** stop immediately, nothing is lost.
- **Exception in any tick:** caught, logged, loop stops with a report; the game continues.
- **Disconnect mid-transfer:** the host has applied every completed step; the state is consistent, with some items moved and the rest on the source.

Shelf identity

A shelf is keyed by its type name plus world position rounded to 0.1 m. This is available on any Transform with no reverse engineering and only breaks if the shelf is physically moved, which this mod exists to avoid. The key lives behind one function so a persistent save ID can replace it later.

SECTION 06

Overlay

The overlay is one panel: what you are moving, where it can go, how many, and a confirm button. It opens on L with the cursor unlocked and game input suppressed; Esc closes it.

Overlay mockup: shelves grouped by base, pick one, set an amount, confirm

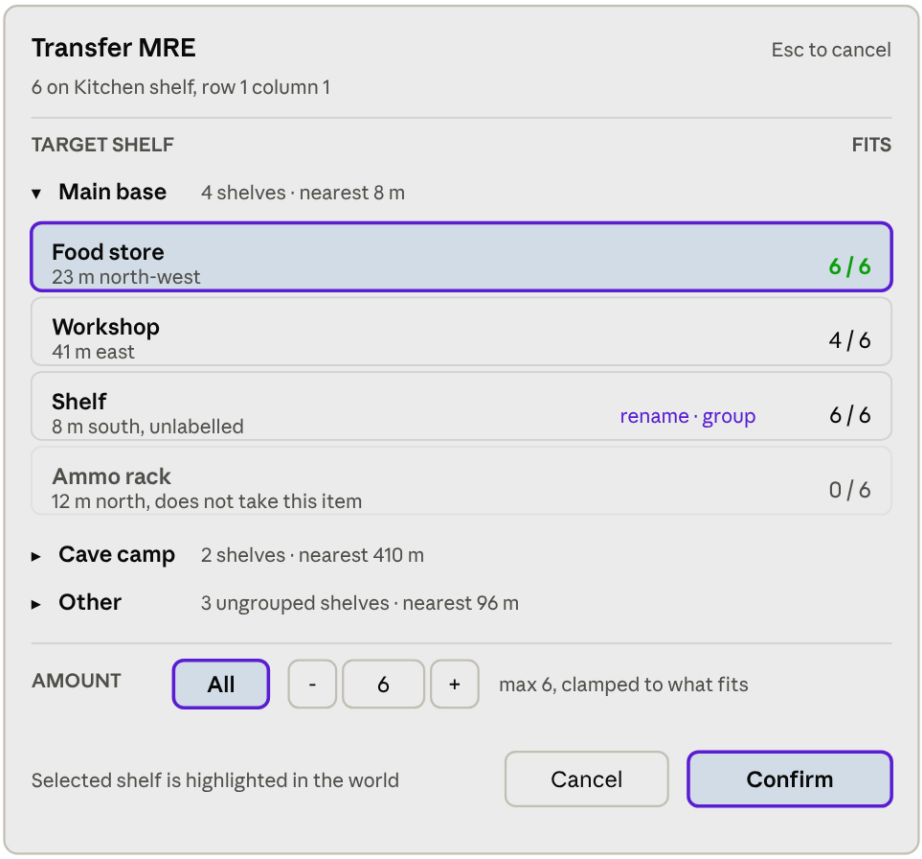


Figure 4. Overlay mockup. Shelves grouped by base, nearest group expanded; the selected shelf is highlighted in the world.

Shelf list

Every shelf on the map except the source, organised in groups. Listing every shelf is not a technical problem: the registry enumerates storage components once when the overlay opens, and the host accepts a store call at any distance. It is a UX problem on a map with several bases, so shelves are grouped by base. Each row shows the label (or "Shelf" plus inline rename and group links when unlabelled), distance and compass bearing, and how many of this item type the shelf can still take. A shelf that cannot hold this item type is dimmed and cannot be selected. Hovering or selecting a row draws a marker on that shelf in the world so you can tell two unlabelled shelves apart.

Groups

A group is a player-defined folder such as Main base or Cave camp, stored with the labels. The overlay shows one collapsible header per group, sorted by distance to the nearest shelf in it, with the nearest group expanded and the others collapsed. Shelves not assigned to any group sit under Other, so a new shelf is always reachable. Within a group, rows are sorted by distance.

Amount

Defaults to All. The stepper and typed value are clamped to the lower of the stack size and the target's free capacity, so a transfer never starts that cannot finish.

Result

After Confirm the panel stays open for a moment showing moved and remaining, then closes. If the transfer stopped early, the panel stays until dismissed and states why.

Rendering

Unity IMGUI drawn from an injected MonoBehaviour, no asset bundles. It will look plainer than the game's own menus; the trade is zero asset tooling and nothing that breaks when the game's UI prefabs change.

SECTION 07

Shelf labels and groups

Labels and groups are client-side names for shelves so the overlay list is readable. The host and other players never see them.

ASPECT	DECISION
Set a label	Look at a shelf and press the label key, or click rename in the overlay; type, press Enter
Group	Optional folder per shelf (Main base, Cave camp). Set from the overlay row or the label editor; new group names are created inline
Shelf key	Shelf type name + world position rounded to 0.1 m, behind one `GetShelfKey()` function
Storage	`BepInEx/config/ShelfTransfer/<worldId>.json`: a map of shelf key to `{ label, group }`
World id	The save's own identifier if readable; otherwise the save slot name
Display	In the overlay list under its group header, and as small world-space text above the shelf when within a few metres
Moved shelf	Loses its label and group (key changes); re-label it. Acceptable because this mod removes the main reason to move a full shelf
Deleted shelf	Entry goes stale; pruned when the registry no longer finds a shelf at that key

SECTION 08

Tech stack

BepInEx 6 IL2CPP was chosen over RedLoader because it is the maintained loader and the mod needs almost none of RedLoader's content SDK.

LAYER	CHOICE	WHY
Loader	BepInEx 6 IL2CPP, build be.755, manual install	Maintained (releases Oct 2024 to Mar 2026); proven by the Admin Menu mod, which does far more invasive things on it. Pinned because newer 6.x builds have broken IL2CPP mods
Patching	HarmonyX (bundled)	One patch at most, on the aim target if it is not readable directly
Game access	`BepInEx/interop` assemblies generated on first launch	Standard Il2CppInterop; regenerate after a game patch
UI	Unity IMGUI from an injected MonoBehaviour	No asset bundles, nothing tied to the game's UI prefabs
Config	BepInEx `.cfg`	Keybinds, items per frame
Labels	Own JSON file	See Section 07
Inspection	UnityExplorer_Sons, BepInEx IL2CPP build	Find the storage class and the take/store methods
Project	C# class library, net6.0, Rider or Visual Studio, post-build copy to `BepInEx/plugins`	

The alternative considered

Runtime speed is identical between the two loaders: both run on .NET 6 with Il2CppInterop and Harmony, and the transfer is bounded by the game's own calls and network round trips. RedLoader offers a UI framework, a test-scene shortcut and game helpers, but its last release was 0.8.6 in February 2025, an open issue reports July 2026 Windows updates stopping it, and it cannot coexist with BepInEx (same Doorstop hook). The game-interop core is written with no loader dependency so it could be re-hosted on RedLoader if that ever becomes necessary.

Sources

- BepInExPack IL2CPP versions — Thunderstore
- Admin Menu (BepInEx) — Nexus Mods
- UnityExplorer_Sons — GitHub
- RedLoader — GitHub
- How to install mods in SotF — modcommunity

SECTION 09

Build order and test plan

Step 2 confirms the whole client-only approach before any UI exists; everything after it is incremental.

1. **Setup.** Install BepInEx be.755, launch once to generate interop, drop in UnityExplorer. In-game, open a shelf and record the storage component's class name, slot layout, item id and count fields, and the methods fired on take and store.
2. **Console command.** `transfer <count>`: take N of the aimed item type from the aimed slot and store into a shelf picked by index, 30 m away. Run on a friend-hosted session; friend opens both shelves. This is the go/no-go for client-only.
3. **Label and group system.** Shelf key, JSON store, label key, group assignment, world-space text. Independent of step 2's outcome.
4. **Transfer hint.** Own-drawn L: Transfer next to the game's E and C prompts when the aim target is a shelf slot.
5. **Overlay.** Grouped shelf list with fits, amount control, confirm, world marker on the selected shelf, result panel.
6. **Acceptance test.** On a friend-hosted session: transfer, friend verifies both shelves, both players save, quit and reload, verify again. Repeat with a full target to check the stop-and-report path.

Test matrix

CASE	EXPECTED
Transfer all, target empty	Source slot empty, target holds N, host agrees
Transfer 3 of 6	Source holds 3, target holds 3
Target can fit 4, ask for 6	Amount clamped to 4 before confirm
Target fills mid-transfer (another player adds items)	Loop stops, report shows moved and remaining, one item at most in inventory
Target shelf of a type that rejects the item	Row dimmed, cannot select
Reload save on both machines	Counts unchanged on both
Esc during overlay	Nothing moves
Mod exception injected in a tick	Logged, loop stops, game keeps running

SECTION 10

Claude Code brief

This document is the source of truth. Claude Code runs the pre-flight checklist with the user, reviews the risks in Section 11 and states how it will handle each before writing any code, then works through the build order one step at a time, updating this document when a decision changes.

Pre-flight checklist

Claude Code checks each item and asks the user to install or confirm anything missing before step 1. It does not download game files or loaders itself.

- [ ] Sons of the Forest installed via Steam on Windows; game folder path confirmed (the folder containing `SonsOfTheForest.exe`)
- [ ] Steam set to update the game only on launch
- [ ] BepInEx 6 IL2CPP build be.755 extracted into the game folder (`winhttp.dll` next to the exe); game launched once; `BepInEx/interop` exists and `BepInEx/LogOutput.log` shows BepInEx loaded
- [ ] No RedLoader files present (`version.dll`, `_RedLoader`, `dotnet`, `doorstop_config.ini` from RedLoader)
- [ ] UnityExplorer_Sons BepInEx IL2CPP build in `BepInEx/plugins`
- [ ] .NET 6 SDK installed (`dotnet --list-sdks`)
- [ ] Rider or Visual Studio 2022 with C# workload
- [ ] A throwaway single-player save for testing, and a friend available for the step 2 and step 6 multiplayer tests
- [ ] Storage class, slot fields and take/store method names recorded from UnityExplorer (step 1 output, added to the open questions in Section 11)
- [ ] **Risk review done:** Claude Code has read the risk table in Section 11, named the mitigation it will apply for each risk in the code, and confirmed the user has a throwaway save so the first transfers never touch a real world

Kick-off prompt

Paste into Claude Code in an empty project folder, with the game folder path filled in.

```
Read the feature doc at <link to this doc or exported PDF> and treat it as the source
of truth for scope, architecture, tech stack and build order.

Before writing any code, run the pre-flight checklist from the "Claude Code brief"
section with me: check what you can from the file system (BepInEx at <game folder>,
interop folder, .NET SDK, plugins) and ask me to confirm or install the rest. Stop and
wait if anything is missing. Then read "Risks and open questions" and, for each risk,
tell me in one line how your implementation will handle it (where the verification
lives, which calls get null checks, which single Harmony patch you expect to need). Do
not start coding until I have acknowledged that list.

Then work through "Build order and test plan" one step at a time. For each step: say
what you are about to do, do it, tell me what to test in game and what result you
expect, then wait for my result before moving on. Step 2 is a go/no-go gate; do not
start UI work until I confirm the host saw the transferred items.

Constraints: BepInEx 6 IL2CPP be.755, HarmonyX with at most one patch, IMGUI UI,
client-only, no host-side code, no direct writes to shelf storage state (only the
game's own take and store calls), one item per frame with verification after each call,
try/catch around every coroutine tick, null and destroyed checks before every interop
call. Keep the game-interop core free of BepInEx types so it could be re-hosted on
another loader.

Project: C# class library ShelfTransfer, net6.0, referencing BepInEx/core and
BepInEx/interop assemblies from <game folder>, post-build copy of the DLL to <game
folder>/BepInEx/plugins. Labels and groups in
BepInEx/config/ShelfTransfer/<worldId>.json.

When a decision in the doc turns out to be wrong against the real game code, say so,
propose the change, and record it in the doc's open questions before proceeding.
```

Working agreement

Claude Code cannot run the game, so every step ends with a test the user performs in game and reports back. Claude Code keeps a short running log in the project README of what was built, what was tested and the result, so a new session can pick up without the original chat.

SECTION 11

Risks and open questions

The one risk that could change the design is whether the host accepts a store call for a distant shelf; everything else is bounded.

RISK	LIKELIHOOD	IMPACT	MITIGATION
Host rejects store on a shelf the player is not near	LOW	Blocks client-only; would need a host-side companion or a single-player-only scope	Step 2 tests it before any UI is built
Game patch breaks BepInEx or interop	MEDIUM	Mod unusable until loader updates	Pin be.755, set Steam to update only on launch, regenerate interop after a patch
Mod bug dupes or deletes items on the host's save	LOW	Real damage to friend's world	Verify after every take and store; test on a throwaway save first
IL2CPP interop quirks (collected objects, missing reflection)	MEDIUM	Slower development, possible hard crash	Null and destroyed checks before every interop call; keep Harmony to one patch
Aim target not readable without a patch	MEDIUM	One Harmony patch needed	Acceptable; the patch is read-only
Position-based shelf key loses labels when a shelf moves	CERTAIN WHEN IT HAPPENS	Re-label a shelf	Swap to a persistent save ID if one is found during step 1

Open questions

- [ ] Exact storage component class and take/store method names (step 1)
- [ ] Does the game expose the aimed slot's row and column directly, or only the item?
- [ ] Is there a persistent per-structure save ID readable from the client?
- [ ] What identifies the current world or save from the client, for the labels file name?
- [ ] Does the host apply a store call for a shelf 30 m away? (step 2)